Supplementary material: Capping realized attention displacement

1 Protocol, deviations and freeze addendum

The binding protocol is `PROTOCOL.md` (SHA-256 `0b0faeb49738938dc625d258bcf7febfdf13e2c1923f24fcba0e56c06fd43069`), supplied as a separate file together with `FREEZE.json` (SHA-256 `937772c096c4a91f9d128ae45888e486d325210eb368ffece2edcba2fbe23930`).

1.1 Deviations

- `2026-09-25T16:00:56` gate: G1; result: failed after the single prespecified fallback (500k tokens, 16 epochs); decision: proceed with the fallback regime (S2_base2); no further unregistered changes; disclosed as a protocol deviation: the S2 regime is multi-epoch data-limited but did not meet the 0.02-nat overfitting criterion
- `2026-09-27T18:30:14` rule: PROTOCOL B11 manipulation check; issue: the outcome-blind B11 check (HB/HF and HB/HU ratios of time-averaged realized mean displacement on the tuning units) was not implemented before the S1 finals were declared; it was noticed and run while S1 finals were in progress (30 of 108 complete); result: {'HB/HF': 0.9330418028326636, 'HB/HU': 0.9691229938532757, 'band': [0.8, 1.25], 'recalibrate': False}; record: studies/S1/manipulation_B11.json; record_sha256: 28a98dc67a1e66056abdf4530c978e7733a00dc452fe2613b262f66fe96fa534; inputs: budget_trace fields of the S1 tuning run records only (realized TV per layer and epoch); no dev or test metric read; consequence: none for the S1 finals: the rule would not have triggered recalibration, so the finals use the parameters the protocol prescribes; the deviation is only in timing; fix: select.py levels now runs the check (manipulation_check.py) before any finals are declared and stops if recalibration is required; S2 will be checked on time
- `2026-09-29T11:07:14` rule: engineering deviation (compute); issue: the RB (relaxed_budget) LM arm does not fit the 3.21 GiB of free VRAM at batch 32; it was not benchmarked before the freeze; decision: run RB and E3 DB LM arms with per-block activation checkpointing; mathematically and numerically identical on the probe (bit-identical loss and gradients); wall-clock for these arms is not comparable and is reported from the matched eager benchmark only; record: results/lm_checkpoint_probe.json
- `2026-09-29T11:22:56` rule: implementation bug (arm specification not applied); issue: S2 DK (DropKey with linear depth schedule) ran without the schedule in 3 tuning runs; the S2 level choice and C*_L were computed from them; decision: exclude the 3 runs (reported, never analysed), rerun them with the fixed runner, and re-apply the prespecified selection rule. The superseded decision (DK@0.20 selected, C*_L = DK@0.20) is kept in excluded/ and disclosed. No test outcome was seen.; record: excluded/S2_DK_noschedule/README.json
- `2026-10-15T10:52:44` rule: PROTOCOL A section 4 hard cap (90 GPU-h or 12 days after the freeze); issue: the 12-day calendar limit (freeze 2026-09-25T15:08:58Z, so the limit was 2026-10-07T15:08:58Z) expired while the campaign was paused at the authors request; the last completed run before the limit finished at 2026-10-04T08:22:15.047721+00:00. Remaining declared extensions: E-S3 phase L (34 of 50 runs), E5, E4, E1, E3.; decision: the authors extended the calendar limit; the 90 GPU-hour cap remains binding and every extension still starts only if its full cost fits. The limit was exceeded through wall-clock pauses (laptop availability), not extra compute.; outcome_blind_evidence: {'unsealed_json_exists': False, 'cv_report_exists': False, 'gpu_hours_used_at_decision': 56.23}; disclosure: stated in the paper (Limitations, Deviations); runs completed after the calendar limit are identifiable from their ledger timestamps

1.2 Freeze addendum (engineering and provenance records, in order)

- `2026-09-25T17:44:31` engineering change after freeze (no change to any protocol rule, arm, recipe or analysis). ['env.boost_process(): process-local HIGH priority and power-throttling opt-out, because Windows ran background runs on efficiency cores (GPU util ~35% -> ~97% after the change; runner throughput ~12 ex/s)', 'encoder_runner: loss/grad-norm accumulated on GPU and flushed every 25 steps (no per-microbatch host sync); non-finite check therefore delayed by <= 25 steps (run still fails and is kept as failed)'] *Files:* env.py b461f0ddf356, encoder_runner.py 603df9e18601
- `2026-09-25T18:58:25` engineering fix after freeze. ['env.boost_process(): declare ctypes HANDLE types; previous version silently failed (64-bit pseudo-handle truncated), so S1 N folds 5-9 and the two gate runs ran at Normal priority (slower wall-clock only). Running tuning processes were raised to High manually.'] *Files:* env.py 596fe6b98683
- `2026-09-25T21:17:13` bug fix after freeze (numerical; no change to any value or gradient where finite). ['attnreg.capped_alpha(): alpha=min(1,tau/r) evaluated as tau/r only where r>tau, else 1. The previous clamp form gave 0*inf=NaN in backward on rows with tiny but nonzero deleted mass. Forward values are identical; gradients identical wherever the old form was finite. Any occurrence produced NaN, so completed runs were unaffected (finite-check passed).', 'tests: TinyDeletedMassGradient (regression + equality with min rule)'] *Files:* attnreg.py e6c3da8196c0
- `2026-09-25T21:31:47` engineering (no effect on results). ['runners exit cleanly before the next run if unified/STOP exists'] *Files:* encoder_runner.py b8c13e2d21cf, lm_runner.py 1994faeaebec
- `2026-09-27T07:56:41` analysis scripts written after the freeze (PROTOCOL B15): hashes recorded before first use and before any unseal. *Files:* stats_unified.py 15b9066033a5, analysis_io.py 6fa961c94c01, cv_report.py a0b3d8dfa19f, unseal.py b2957a255fdc, analyze.py c607641f5dd6, select.py e736d7f2e7c3, campaign.py a61cade81265, drive.sh e4459b44ee94, tests/test_stats_unified.py d2350f4fe1d0, tests/test_analyze.py e28f9d526098
- `2026-09-27T07:58:06` test-only change. sklearn cross-check replaced by exact pairwise AUROC (venv numpy/sklearn incompatibility); analysis_io excludes superseded S2_base *Files:* tests/test_stats_unified.py 19e2293b85fb, analysis_io.py 6fa961c94c01
- `2026-09-27T08:12:17` provenance correction + extension features (no change to any declared core run). runners computed source_sha256 when writing each result, not at process start. Runs executing across an edit therefore record the on-disk hash, not the executed code. Affected so far: S1/n folds 4-9 (executed the pre-sync-fix loop and pre-boost-fix env.py), and S1/finals runs completed after this record (they execute the code in executed_sha256 below; their recorded encoder_runner.py hash is the new one). ['source hashes now computed once at process start (encoder_runner, lm_runner)', 'encoder_runner: eval_pool, soft_targets KD (T^2 KL from a teacher run export, hash-checked), ce_weight=0 for unlabeled transfer sets, per-spec test_inputs/test_donors (extensions E-S3, E4)'] *Files:* encoder_runner.py 03a0a0057120, lm_runner.py 7da84a51e6cb, s0_census.py 04b08eef6142, prepare_biolinkbert.py 0fa5f76c2836, campaign.py 78d6e13feef8
- `2026-09-27T08:13:47` extension data (E4, E5). *Files:* env.py d2f083b55e45, prepare_scifact.py ebed6b24f751, data/scifact/manifest.json f2de45663422, models/biolinkbert_base/download_manifest.json 8dfd777b79e6
- `2026-09-27T08:17:32` extension stage declarations and drivers (PROTOCOL B8), recorded before first use. *Files:* campaign.py 491a2f3db96a, select.py 77faa56fa35e, s0_census.py d74535fcea69, unseal.py d9687452dd2f, smoke_ext.py 36e5df15e51a, drive_ext.sh abf68bdab554, encoder_runner.py 03a0a0057120, env.py d2f083b55e45, prepare_scifact.py ebed6b24f751
- `2026-09-27T17:35:45` E1/E3 declarations and driver, recorded before first use. E1 alpha selected on folds 0-1 by lowest mean dev Brier (ties -> lower alpha); E3 dropkey_budget p=0.5, tau calibrated (predictions_e3.json) *Files:* campaign.py 20d51f76d07c, calibrate.py 23ecce49232d, drive_ext2.sh 75f272aac7c7, results/predictions_e3.json 20032f362a7a
- `2026-09-27T18:30:14` B11 manipulation check implemented (outcome-blind). select.py levels calls manipulation_check.check before level selection; S1 record written post hoc (see DEVIATIONS.jsonl); no change to any run specification *Files:* manipulation_check.py 9a1ce5a45448, select.py de812b330b61

- 2026-09-27T18:37:59 audits implemented (PROTOCOL A section 4 audit row and section 5 audit.py; B14 R14), recorded before first use. audit.py was planned but not written before the freeze. Audit runs are chosen by sha256(FREEZE sha + run_id) after the core. R14 asks for deterministic training of both the original and the replay; the core runs were trained non-deterministically, so each chosen spec is trained twice in deterministic mode (_detA/_detB, compared within 1e-6 and by sealed hash) and _detA is compared with the original within the N between-unit SD (dev metrics only). Evaluation replays use the saved checkpoints. Audit runs are outside every analysis. drive_ext.sh runs the audits after the core and before the extensions (non-fatal on failure). *Files:* audit.py 212220fec15d, drive_ext.sh 9241f4ecbccc
- 2026-09-27T18:45:47 extension analyses and pre-unseal fixes, recorded before any unseal. ['analyze_ext.py (new): E2 failure ladder, E4/E5 replication, E-S3 gates G3/G4 with retention/agreement/cost, E1, E3, B6 folds 2-9 replication, P1 seed-2 H1-E, exploratory (Brown-Forsythe, dev dose-response, DAM, LM frequency buckets), venue facts (B5)', 'analyze.py: F2 gains S-a/S-b only if G3 and G4 pass (B1, B7); F2 verdicts use the F1 rules with the F2 Holm p; results include extensions', 'unseal.py: audit replays excluded from scoring (their run ids are not analysis units); per-condition prediction shares added (E4 NEI shift under rationale deletion)', 'campaign.py e1_finals: sealed test files of unselected R-Drop alphas deleted unopened (B12.3), listed in decisions_E1.json'] see analyze_ext.py docstring: B5 replication assessed on HB@0.10-HU@0.10 in E4/E5 with the H1-E rules (unadjusted); E2 upper 90% bound = upper limit of two-sided 90% fold-cluster bootstrap; extension contrasts other than S-a/S-b unadjusted and descriptive *Files:* analyze.py 26a3992db1d4, analyze_ext.py 89f370ffc06c, unseal.py b416c3d4c39f, campaign.py 6f1c21cb4c87, audit.py 212220fec15d
- 2026-09-27T18:54:16 figure code recorded before any unseal. figures_census.py is outcome-independent (census + predictions only; output results/census_layers.json, paper/figures/fig_census.pdf). figures_results.py (F2-F6) reads analysis/unsealed/cv outputs and will run once after the unseal; tested on synthetic data only. *Files:* figures_census.py a31aafe8cb5d, figures_results.py ecb2d4507539
- 2026-09-29T11:07:14 engineering: per-block activation checkpointing for LM arms that exceed free VRAM (outcome-blind). S2_RB@0.10_s1 hit CUDA OOM at step 0 (3.21 GiB free; RB needs 3.28 GiB with optimizer state). RB was never in the pre-freeze LM benchmark. lm_runner now wraps each block in torch.utils.checkpoint (non-reentrant, RNG state preserved) when arm kind is relaxed_budget or dropkey_budget (E3 DB also OOMs). Probe on random tokens: loss and all gradients bit-identical with and without checkpointing (max |diff| 0), realized TV identical; only memory and wall-clock change. All other arms unchanged. Budget statistics are accumulated during recomputation as well, which scales rows and sums equally and leaves every reported average unchanged. *Files:* lm_runner.py cbdfab688d4c, probe_lm_checkpoint.py 08ea979175fd, results/lm_checkpoint_probe.json 5ccb72f7a7f5
- 2026-09-29T11:22:56 bug fix: LM DropKey depth schedule (outcome-blind). lm_runner ignored arm.layer_schedule, so LM DK runs used p_max in all six layers. Found from the B11 manipulation record (DK realized D 0.086/0.171/0.343 vs 0.05/0.10/0.20), which shows no outcome. The encoder runner was correct (S1, E-S3, E5 and E4 are unaffected); E3 DB has no schedule. Fix: the factory applies attnreg.depth_schedule as in encoder_runner. Smoke on seed 999: per-layer realized D equals p_max(1-i/6) within 1e-3, pooled 0.100. Remediation: the 3 buggy S2 DK tuning runs, the unopened DK@0.20 sealed files, and the superseded S2 level decision and finals registry were moved to excluded/S2_DK_noschedule. The 3 DK tuning runs are rerun with the same seeds, then the unchanged selection rule is re-applied and the finals are redeclared. S2_AD@0.20_s2 is kept (identical spec, unaffected). *Files:* lm_runner.py 2e422e53e108, remediate_dk_lm.py cb26882834c7, excluded/S2_DK_noschedule/README.json 052fb500fdb8
- 2026-09-29T11:42:22 DK remediation outcome (outcome-blind). rerun DK tuning realized D 0.050/0.100/0.200 (targets 0.05/0.10/0.20). The re-applied selection rule gives the same decision as the superseded one (DK 0.20; C*_L = DK@0.20; RA* 0.05). The redeclared S2 finals registry is identical to the superseded registry (99 runs), so S2_AD@0.20_s2 stands. *Files:* studies/S2/decisions_levels.json 11bbae18a7b3, studies/S2/finals/registry.json e20ce7abf57c
- 2026-09-30T14:26:48 audit.py report fix before its first use. encoder sealed test files embed their run_id, so _detA/_detB raw hashes always differ. report() now checks each file against its recorded hash and compares A (run_id swapped for B) byte-for-byte with B, in-process, keeping only a boolean.

run() is unchanged. LM sealed files contain no run_id and keep the raw-hash comparison. *Files:* audit.py 2c44146cb0b7

- `2026-10-01T06:52:29` audit evaluation-replay fix (procedure, not data). v1 encoder evaluation replay of S1_LS@0.2_f3_detA failed (dev max |diff| 0.056) because the replay used default SDPA kernels while that run, trained under audit.deterministic(), evaluated with MATH-only SDPA. evalreplay now sets the backend the run used (_detA: MATH only; originals: defaults). v2: all three encoder replays are bit-exact (dev diff 0, sealed hash equal). LM replays passed in v1. v1 files are kept in results/superseded and studies/audit/superseded. Retrain audits: all four runs pass both the deterministic and the non-deterministic criteria. *Files:* audit.py aa563a26530f, results/audit.json e4984544225b, results/audit_evalreplay_encoder.json 2e6afb133f82

- `2026-11-02T06:33:08` engineering: thermal governor and compute-time budget accounting (outcome-blind; no effect on computation). At the authors request (laptop overheating), thermal_governor.py suspends experiment processes when the GPU reaches 70 C and resumes them at 60 C. Suspension stretches wall time only: seeds, data order, counter-based noise and kernels are unchanged, and no runner logic depends on elapsed time (ThroughputGuard is defined but unused). Each pause is logged in studies/thermal_pauses.jsonl. budget.py computes cap usage as ledger wall time minus the overlap of pauses with each ledgered run, and the drivers fits() now call budget.py, so pauses do not consume the 90 GPU-hour cap. Run records written after this point include pause time in train_seconds; wall-clock cost is reported from the matched benchmark and measure_cost.py, not from run records. Suspend/resume verified on a dummy process. *Files:* thermal_governor.py b9f4d6a3817f, budget.py 6b16d429b103, drive_ext.sh f42a8c2b2a04, drive_ext2.sh 57fc03403fff

- `2026-11-02T06:36:46` thermal governor v2: duty cycle (engineering only). v1 pauses lasted ~3 s, because the GPU core cools quickly, so average heat barely fell. v2 adds a duty cycle: after 60 s of work the experiment processes are suspended for at least 60 s and resumed only at <= 60 C. The 70 C ceiling remains. Average GPU power is at most half of full speed and wall time is about 2x or more. The process list is refreshed every 15 s instead of every poll. Pause reasons (duty or thermal) are logged. No effect on computation. *Files:* thermal_governor.py 4cebe64d2be4

- `2026-11-02T06:37:27` thermal governor v3 (engineering only). the GPU reaches 70 C about 10 s after each resume, so thermal pauses pre-empted the 60 s duty cycle and the GPU worked about 60% of the time. v3: every rest lasts at least as long as the work stretch before it (and until <= 60 C), so the duty cycle is at most 50%. No effect on computation. *Files:* thermal_governor.py 35d06b189206

- `2026-11-02T14:30:20` engineering: drive_ext.sh interpreter fix (no effect on computation). At 2026-11-02T14:29Z, after ES3 phase L completed (50/50), drive_ext.sh stopped before E5: it ran 's0_census.py calibrate_only biolinkbert e5' with the system Python, which lacks transformers (ModuleNotFoundError at import). Nothing was written: results/predictions_e5.json and studies/E5 do not exist. The line now uses the project venv, as the student-census line already did. calibrate_only only calibrates the existing results/s0/biolinkbert_census.json (written 2026-09-27); the E5 declarations and runs are unchanged. *Files:* drive_ext.sh ae2268ac81cd

- `2026-11-08T20:15:41` extension decisions after E3: E6 not run (budget); E8 implementation and analysis plan (recorded before E8 is declared or run, and before the unseal). *Files:* attnreg.py a13a8e73a0c4, campaign.py f96eb04f4af0, analyze_ext.py 310f7573e4ba, tests/test_e8.py b7409ccec9a9, drive_e8.sh 0ccf80530533 {"previous_sha256": {"attnreg.py": "e6c3da8196c0ab9791061e80790d4bf8aeafcb3239356bba8eea112c44345324", "campaign.py": "6f1c21cb4c877fe4c4ffe666ac06160293168c82e316c12977c4be688672c99d", "analyze_ext.py": "89f370ffc06c8fedb3ee0e1b2341b0938cabbb5582ba538b0254b7f242f34dea"}}

- `2026-11-08T22:38:51` cv_report recorded before the single unseal (PROTOCOL A 3.0). all 22 registries complete (0 pending), incl. E8 (S2/e8, 10/10); E6 not run (budget); compute 86.27/90 GPU-h; results/cost.json written before this record {"cv_report_sha256": "0a168e1767ec3f645038c2f13a6bc68121c327822b99eaac463c84740e4ba279"}

2 Run inventory

Every declared run of every study, its status and its GPU time from the study ledger.

Study	declared	completed	pending	GPU-h
E2	60	60	0	2.30
E4	40	40	0	16.45
E5	40	40	0	16.82
ES3/phase_k	10	10	0	0.47
ES3/phase_l	50	50	0	9.43
ES3/teachers	10	10	0	1.19
S1/e1_finals	8	8	0	5.39
S1/e1_tuning	6	6	0	4.12
S1/e3	10	10	0	5.66
S1/finals	108	108	0	19.83
S1/gate	2	2	0	0.64
S1/n	10	10	0	1.34
S1/p1	20	20	0	5.54
S1/tuning	44	44	0	8.96
S2/e3	10	10	0	3.40
S2/e8	10	10	0	2.36
S2/finals	99	99	0	10.37
S2/tuning	19	19	0	2.27
S2_base2	3	3	0	0.28
S2_base (superseded)	3	3	0	0.28
audit/S1	4	4	0	0.76
audit/S2	4	4	0	0.38
Total				118.24

Excluded runs Kept and reported but never analysed; their sealed files were never opened.

- S2_DK_noschedule: One-off remediation (2026-09-29) of the LM DropKey implementation bug: lm_runner ignored the DK arm's 'layer_schedule' ("dropkey_linear"), so the three S2 DK tuning runs used p_max in every layer (realized mean displacement ~1.7x the calibrated level). Outcome-blind: no test file is opened. Moved: runs/S2_DK@0.05_s1.json, runs/S2_DK@0.10_s1.json, runs/S2_DK@0.20_s1.json, sealed/S2_DK@0.20_s1_final.json, sealed/S2_DK@0.20_s1_best.json, decisions_levels_superseded.json, finals_registry_superseded.json.

3 Incidents

- E5 2026-11-03T15:00:57 E5_AD@0.20_f4: run stopped mid-training (about 5 of 22 minutes) at the authors request because the laptop was being closed; no result, sealed or partial file written; rerun once from scratch with the same seed on resume (files deleted unopened: none)
- ES3/phase_l 2026-10-15T11:10:41 ES3_S-HU@0.10_f3: run stopped mid-training (about 3 of 8 minutes) at the authors request because the laptop was overheating without air conditioning; no result, sealed or partial file written; rerun once from scratch with the same seed on resume (files deleted unopened: none)
- S1/e3 2026-11-07T16:41:00 S1_DB@0.10_f3: thermal governor process ended with the assistant session at about 16:38Z while it had the run suspended; the run stayed suspended (GPU idle, 46 C) until the governor was restarted at 16:40:15Z and resumed it. Suspension only delays wall-clock time; the run continued unchanged with the same seed. Pause appended to thermal_pauses.jsonl. Governor now launched detached. (files deleted unopened: none)

- S1/n 2026-09-25T16:00:56 S1_N_f0: process killed by host memory-pressure reaper after sealed test file and checkpoint were written, before result JSON; one logged rerun with the same seed per PROTOCOL A 3.0 (files deleted unopened: none)
- S1/tuning 2026-09-25T21:17:13 S1_HU@0.20_f0: non-finite gradient caused by an implementation bug (see FREEZE_ADDENDUM), not by the method; rerun once with the same seed after the fix (files deleted unopened: none)
- S2/tuning 2026-09-29T09:13:24 S2_AD@0.05_s1: duplicate driver chain: after a Claude Code memory-reaper event the original drive.sh survived unmanaged and a restarted drive.sh also launched lm_runner on studies/S2/tuning; two processes trained S2_AD@0.05_s1 concurrently from 09:11:27/09:11:35 UTC. The earlier (unmanaged) chain was killed at ~09:12:40 UTC before it wrote any result, sealed or census file; the retained process continues the run. gpu_log.csv rows 09:11:27-09:12:40 interleave both processes (descriptive telemetry only). The retained run shared the GPU for ~75 s, which affects its wall-clock only. (files deleted unopened: none)
- S2/tuning 2026-09-29T11:07:14 S2_RB@0.10_s1: CUDA out of memory at training step 0 (before any output); no result, sealed, census or ledger entry written; rerun once with the same seed after the checkpointing disposition (files deleted unopened: none)
- S2/tuning 2026-09-29T11:22:56 S2_DK@0.05_s1,S2_DK@0.10_s1,S2_DK@0.20_s1: implementation bug (depth schedule ignored); results moved unopened to excluded/S2_DK_noschedule; rerun with the same seeds after the fix (files deleted unopened: none)

4 Audits

- S1_LS@0.2_f3: deterministic replay max |diff| 0.00e+00, sealed hashes equal True (pass); original vs replay |dev diff| 0.0105 against tolerance 0.1368 (pass)
- S1_RA@0.10_f2: deterministic replay max |diff| 0.00e+00, sealed hashes equal True (pass); original vs replay |dev diff| 0.0000 against tolerance 0.1368 (pass)
- S2_N_s6: deterministic replay max |diff| 0.00e+00, sealed hashes equal True (pass); original vs replay |dev diff| 0.0000 against tolerance 0.0028 (pass)
- S2_N_s3: deterministic replay max |diff| 0.00e+00, sealed hashes equal True (pass); original vs replay |dev diff| 0.0000 against tolerance 0.0028 (pass)
- evaluation replay (encoder) S1_N_f0: {"dev_max_abs_diff": 0.0, "sealed_hash_equal": true}
- evaluation replay (encoder) S1_LS@0.2_f3_detA: {"dev_max_abs_diff": 0.0, "sealed_hash_equal": true}
- evaluation replay (encoder) S1_RA@0.10_f2_detA: {"dev_max_abs_diff": 0.0, "sealed_hash_equal": true}
- evaluation replay (lm) S2_base2_N_lr5e-4_s1: {"replays_bit_identical": true, "val_nll_abs_diff_vs_record": 1.5166584841352915e-06}
- evaluation replay (lm) S2_N_s3_detA: {"replays_bit_identical": true, "val_nll_abs_diff_vs_record": 2.1629966138192458e-05}
- evaluation replay (lm) S2_N_s6_detA: {"replays_bit_identical": true, "val_nll_abs_diff_vs_record": 2.179048224615343e-05}

5 Per-layer displacement (PROTOCOL B10)

Census prediction, encoder, $\ell = 0.10$

layer	1	2	3	4	5	6	7	8	9	10	11	12
HU $\mathbb{E}d$	0.041	0.065	0.072	0.074	0.064	0.097	0.143	0.151	0.118	0.115	0.131	0.130
HF $\mathbb{E}d$	0.041	0.065	0.072	0.074	0.064	0.097	0.143	0.151	0.118	0.115	0.131	0.130
HB $\mathbb{E}d$	0.056	0.074	0.083	0.093	0.092	0.100	0.104	0.116	0.116	0.120	0.123	0.122
HB $\Pr(m > \tau)$	0.17	0.25	0.34	0.42	0.39	0.50	0.64	0.76	0.70	0.74	0.80	0.80

Census prediction, lm, $\ell = 0.10$

layer	1	2	3	4	5	6
HU $\mathbb{E}d$	0.062	0.090	0.102	0.119	0.124	0.103
HF $\mathbb{E}d$	0.062	0.089	0.102	0.119	0.125	0.103
HB $\mathbb{E}d$	0.076	0.095	0.101	0.112	0.112	0.103
HB $\Pr(m > \tau)$	0.14	0.28	0.34	0.41	0.42	0.33

Realized during training Mean realized displacement $\bar{D}$ per layer, row-weighted over all epochs and units of each arm (training-mode rows, recorded on the GPU).

regime	arm	$\bar{D}$ by layer
S1	AD@0.05	0.050 0.050 0.050 0.050 0.050 0.050 0.050 0.050 0.050 0.050 0.050 0.050
S1	AD@0.10	0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100
S1	AD@0.20	0.200 0.200 0.200 0.200 0.200 0.200 0.200 0.200 0.200 0.200 0.200 0.200
S1	DB@0.10	0.102 0.099 0.102 0.103 0.103 0.098 0.091 0.098 0.101 0.102 0.100 0.100
S1	DK@0.05	0.092 0.085 0.077 0.069 0.062 0.054 0.046 0.038 0.031 0.023 0.015 0.008
S1	DK@0.10	0.185 0.169 0.154 0.138 0.123 0.108 0.092 0.077 0.062 0.046 0.031 0.015
S1	DK@0.20	0.369 0.338 0.308 0.277 0.246 0.215 0.185 0.154 0.123 0.092 0.062 0.031
S1	HB@0.05	0.040 0.044 0.047 0.050 0.050 0.051 0.049 0.053 0.054 0.054 0.054 0.054
S1	HB@0.10	0.059 0.072 0.081 0.091 0.090 0.099 0.105 0.116 0.114 0.117 0.121 0.119
S1	HB@0.20	0.101 0.124 0.144 0.164 0.160 0.187 0.229 0.246 0.233 0.242 0.256 0.256
S1	HF@0.10	0.041 0.059 0.067 0.072 0.063 0.097 0.146 0.148 0.121 0.117 0.148 0.148
S1	HU@0.05	0.020 0.030 0.034 0.036 0.031 0.048 0.073 0.075 0.061 0.059 0.076 0.076
S1	HU@0.10	0.041 0.058 0.066 0.071 0.062 0.095 0.144 0.147 0.120 0.115 0.148 0.147
S1	HU@0.20	0.082 0.115 0.129 0.135 0.119 0.182 0.284 0.288 0.235 0.231 0.306 0.306
S1	L0	0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100
S1	RA@0.05	0.029 0.036 0.038 0.042 0.042 0.050 0.060 0.061 0.057 0.057 0.062 0.062
S1	RA@0.10	0.059 0.071 0.075 0.083 0.083 0.099 0.120 0.122 0.113 0.114 0.125 0.124
S1	RA@0.20	0.118 0.140 0.147 0.162 0.164 0.193 0.237 0.240 0.224 0.225 0.251 0.249
S1	RB@0.10	0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100
S1	RDROP@0.1	0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100
S1	RDROP@0.5	0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100
S1	RDROP@1.0	0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100 0.100
S2	AD@0.05	0.050 0.050 0.050 0.050 0.050 0.050
S2	AD@0.10	0.100 0.100 0.100 0.100 0.100 0.100
S2	AD@0.20	0.200 0.200 0.200 0.200 0.200 0.200
S2	DB@0.10	0.100 0.100 0.100 0.100 0.100 0.100
S2	DK@0.05	0.086 0.071 0.057 0.043 0.029 0.014
S2	DK@0.10	0.171 0.143 0.114 0.086 0.057 0.029
S2	DK@0.20	0.343 0.286 0.229 0.172 0.114 0.057
S2	HB@0.05	0.045 0.049 0.053 0.057 0.056 0.055
S2	HB@0.10	0.092 0.096 0.113 0.124 0.127 0.117
S2	HBsg@0.10	0.084 0.093 0.108 0.121 0.124 0.115
S2	HF@0.10	0.079 0.105 0.122 0.147 0.149 0.131

regime	arm	$\bar{D}$ by layer
S2	HU@0.05	0.033 0.045 0.054 0.064 0.068 0.058
S2	HU@0.10	0.071 0.093 0.115 0.133 0.139 0.122
S2	HU@0.20	0.155 0.194 0.263 0.308 0.323 0.264
S2	RA@0.05	0.030 0.049 0.061 0.066 0.072 0.064
S2	RA@0.10	0.063 0.106 0.127 0.146 0.160 0.149
S2	RA@0.20	0.142 0.282 0.267 0.371 0.429 0.386
S2	RB@0.10	0.082 0.094 0.095 0.094 0.094 0.094

6 Literature search

The prior-art search and its adversarial refutation round are archived as `design/novelty/` (claim table, per-claim refuter reports and the synthesis). Coverage limits are stated in the main text.